

30224 – Information Systems

Practice 5: Deployment of a Web Information System with data persistence by using Docker containers

2025-11-25

Table of contents

1	Objectives	2
2	Contents	3
2.1	Lab agenda (recommended order of tasks)	4
3	Docker Installation	4
3.1	Understanding Docker Concepts	4
3.2	Basic Docker Commands	5
4	Identification and preparation of the elements	5
4.1	Prerequisites	5
4.2	Application Server Setup	6
4.2.1	Tomcat	6
4.2.2	Node.js	9
4.3	Database Setup	10
4.3.1	Local Database (PostgreSQL)	10
4.3.2	Cloud Database (Option 2)	12
5	Image Construction and Test of the System	12
5.1	Prerequisites	12

5.2	Building the Application Server Image	12
5.2.1	Tomcat Image	12
5.2.2	Node.js Image	14
5.3	Building the Database Image	15
5.4	Running the Containers	16
5.4.1	Starting the Database Container	16
5.4.2	Starting the Application Server Container	17
5.4.3	Verifying the Deployment	18
5.5	Creating a Deployment Script	19
5.6	Test of the System	20
5.6.1	Testing Procedure	20
5.6.2	Common Issues and Solutions	21
6	Practice submission	22
6.1	Deadline and submission channel	23
7	Grading criteria	23
7.1	Minimum requirements (7 points)	23
7.2	Additional considerations (3 points)	24
8	Appendix. Optional: Advanced Deployment with Docker Compose and NGINX	24
8.1	Why Use Docker Compose?	24
8.2	Enhanced Architecture with NGINX	25
8.3	Implementation Steps	25
8.3.1	1. Create NGINX Configuration	25
8.3.2	2. Create NGINX Dockerfile	26
8.3.3	3. Create docker-compose.yml	26
8.3.4	4. Update Tomcat Configuration	28
8.3.5	5. Deploy with Docker Compose	28
8.3.6	6. Access the Application	28
8.4	Managing the Deployment	28
8.5	Benefits of This Architecture	29
9	Additional resources	29

1 Objectives

Web applications are applications that run on at least one Web server and are accessed by users from a general-purpose client (for example: a Web browser on a PC, mobile phone, etc.).

Once our application has been designed and developed, it needs to be deployed on a server that runs its different components:

- Application Server
- Database server
- (Optional) http/https server

Thus, the objectives of this practice are:

- **Identification and preparation** of the application components that we must deploy (code, data, static elements, ...).
- **Definition of the deployment model** that we are going to use in order to make our application accessible:
 - Use of Docker Containers
 - Local Database
 - Database on the Cloud
- **Deployment of the application** according to the deployment model chosen.
- **Verification** of the correct execution of the scripts to deploy the web application from scratch (Docker) and verification of the correct operation of the application on the server where it has been deployed.

The environment described in this document assumes an application developed using Jakarta EE on Tomcat, and with PostgreSQL as a database. However, different practice groups may have selected other technologies and tools, so these scripts must be adapted to the selected technology.

i Note

The reference environment includes **Java 21 LTS**, **Jakarta EE 10/11**, **Tomcat 10.1/11**, and **PostgreSQL 17**. Alternatives such as **Nest.js** (Node.js) or **Django** (Python) are acceptable.

2 Contents

The practice is divided into four main blocks:

1. **Docker installation and setup:** Understanding Docker concepts, installation, and basic commands.
2. **Identification and preparation of elements:** Preparing application components (WAR files, database files, configuration files) for deployment.
3. **Image construction and testing:** Creating Docker images and containers, and testing the complete system.
4. **Deployment verification and deliverables:** Testing the deployment process and preparing final deliverables.

2.1 Lab agenda (recommended order of tasks)

Two sessions have been planned for this practice. The following planning is suggested as a guide:

- **First session: Docker installation and identification and preparation** of the application components (WAR file, database files, configuration files). **Construction of Docker images** for Tomcat (or Node.js) and PostgreSQL. **Initial testing** of containers.
- **Second session: Complete deployment testing** on clean systems, **verification** of the deployment scripts, and **preparation of deliverables** (deployment package and practice report).

3 Docker Installation

Docker is software that packages and virtualizes applications. Think of it as a lightweight virtual machine environment. Unlike VirtualBox which virtualizes entire operating systems, Docker works at the application level.

Docker allows us to package our application with Tomcat in one container, the database in another container, and other components in separate containers. These containers can be copied and deployed on any machine, or you can prepare build instructions that work on any server.

The main advantage of Docker is that it avoids version conflicts, missing libraries, or configuration problems. If an application works in a container, it will work on any machine where you deploy that container, because the application and all its dependencies are packaged together.

To install Docker on your system, follow the instructions available on [Docker Get Started](#).

3.1 Understanding Docker Concepts

It is important to distinguish between **images** and **containers** when we use Docker.

Docker Images Templates used to build containers. Images are read-only files that define what will run inside a container.

Docker Containers Running instances created from images. Containers are the actual running applications.

To create a container, you need an image. What actually runs on your computer are containers. You can have multiple containers running from the same image. Each container has its own resources: communication ports, folders, processes, and memory space.

On [Docker Hub](#), you can find many ready-to-use Docker images. You can use these images directly or as a base to build more complex images. For example, you can start with a Tomcat image and add your application on top of it, creating a new image that contains both Tomcat and your application.

3.2 Basic Docker Commands

Some basic Docker commands (see more documentation on [Docker Documentation](#)) are:

```
docker ps                # Show active containers
docker ps -a             # Show active and stopped containers
docker run --name my_container my_image # Create and start/run a new
container from a Docker image
docker start my_container # Start a stopped container
docker stop my_container  # Stop a container
docker rm my_container    # Delete a container
```

Tip

You can use `docker ps` to verify that your containers are running. If a container is not listed, check its status with `docker ps -a` to see stopped containers.

4 Identification and preparation of the elements

Before creating Docker images, we need to prepare all the necessary files for our deployment. This section explains how to gather and organize these components.

4.1 Prerequisites

Before starting, ensure you have:

- Completed Practice 4 and have a working WAR file
- Access to your Tomcat installation (for configuration files)
- Your PostgreSQL database ready (if using local database)
- Docker installed and running (see [Section 3](#))

4.2 Application Server Setup

4.2.1 Tomcat

First, create or identify a folder where we will copy all files needed for our deployment. This folder will be our work folder. We can call this folder “Docker” and create it in the workspace of our IDE.

```
$HOME/eclipse-workspace/sisinf_project/docker
```

The basic elements that we need to deploy the applications are:

- **WAR file** with the source code (static and dynamic content of the web application).
- **Configuration files** (`context.xml` and `server.xml`) from your Tomcat installation.
- **Database driver** (PostgreSQL JDBC driver JAR file).

4.2.1.1 Understanding WAR Files

A WAR (Web Application Archive) file is a package that contains all the files needed for a web application. It includes:

- Compiled Java classes (servlets, JavaBeans, etc.)
- JSP files
- Static resources (HTML, CSS, JavaScript, images)
- Configuration files (`web.xml`, etc.)
- Libraries (JAR files) in `WEB-INF/lib`

In previous practices, a WAR file was created and tested. Any IDE compatible with Jakarta EE (such as Eclipse or IntelliJ) allows us to generate a WAR file with our application.

! Important

The folder `WEB-INF/lib` must include all libraries (JAR files) that our application needs and that are not included in Tomcat. For example, the library for JSP tags (JSTL). In Eclipse, you can configure this with the project properties “Java Build Path” and “Deployment Assembly”.

4.2.1.2 Generating a WAR File

To generate a WAR file in Eclipse:

1. Select the project in the Project Explorer.
2. Right-click and select **Export...** → **WAR file**.
3. Choose the destination folder and click **Finish**.

! Important

If the version of Java with which you are working is different from the version of Java used in the Docker image, you may encounter the following error:

```
java.lang.UnsupportedClassVersionError:
es/unizar/sisinf/grp1/servlet/Signin has been compiled by a more
recent version of the Java Runtime (class file version 58.0)
```

To fix this, you must compile and generate the WAR file with a compatible Java version:

1. Right-click the project and select **Properties**.
2. Select **Java Compiler**.
3. Configure a Java version compatible with the Docker image (e.g., set “Compiler compliance level” to 11).
4. Rebuild and export the WAR file.

4.2.1.3 Preparing the Deployment Folder

Now, prepare your deployment structure:

1. Create a `tomcat` folder inside your `Docker` folder.
2. Copy your WAR file to the `tomcat` folder and rename it to `ROOT.war`:

```
$HOME/eclipse-workspace/sisinf_project/Docker/tomcat/ROOT.war
```

💡 Tip

Renaming the WAR file to `ROOT.war` allows you to access the application without including the project name in the URL. Instead of `http://localhost:8080/my_project`, you can access it at `http://localhost:8080`.

3. Copy the configuration files from your Tomcat installation:

- `context.xml` — from `$TOMCAT_HOME/conf/context.xml`
 - `server.xml` — from `$TOMCAT_HOME/conf/server.xml`
4. Copy the PostgreSQL driver JAR file. The driver should be named something like `postgresql-42.x.x.jar` (the version number may vary). Copy it from your Tomcat `lib` folder or download it from [PostgreSQL JDBC Driver](#).

Caution

If you copied the `server.xml` file directly from your Tomcat installation, it may contain references to applications deployed in your test environment. Look for lines like:

```
<Context docBase="my_project" path="/my_project" ...
```

This line must be deleted, since we are going to deploy the project as `ROOT`. Otherwise, Tomcat will give an error when starting, not finding the original project in its expected location.

4.2.1.4 Docker Network Configuration

Each Docker container works like a separate server in a virtual network. We will deploy Tomcat in one container and PostgreSQL in another container.

Both services run as separate servers, each with its own network address. Containers can communicate with each other, and you can access them from your host machine.

Important

With containers, the database is **not** available at “localhost” for the Tomcat server. “localhost” in the Tomcat container refers to the Tomcat container itself, not your host machine.

Since Tomcat and PostgreSQL run in different containers, they need to communicate using container names. We will name the PostgreSQL container “`sisinf-database`” and the Tomcat container “`sisinf-tomcat`”.

Before creating the image for the Tomcat container, you must change the `server.xml` file to define connections with the PostgreSQL database. Specifically, substitute “localhost” with the database container name in the database connection URL.

If using Docker Compose, use the service name `database`:

```
url="jdbc:postgresql://database:5432/postgres?currentSchema=<_
your_schema>"
```

If using manual Docker commands, use the container name `sisinf-database`:

```
url="jdbc:postgresql://sisinf-database:5432/postgres?currentSchema=<_
your_schema>"
```

Replace `<your_schema>` with your actual database schema name.

 Tip

If you are using a `ConnectionManager` instead of datasources defined in Tomcat, make the same change there: replace “localhost” with `database` (for Docker Compose) or `sisinf-database` (for manual Docker commands) in the connection URL.

Checkpoint: Tomcat Preparation

At this point, you should have:

- A `tomcat` folder inside your `Docker` folder
- `ROOT.war` file in the `tomcat` folder
- `context.xml` and `server.xml` files in the `tomcat` folder
- PostgreSQL driver JAR file in the `tomcat` folder
- `server.xml` modified to use “sisinf-database” (or “database” for Docker Compose) instead of “localhost”

Verify your folder structure:

```
Docker/
  tomcat/
    ROOT.war
    context.xml
    server.xml
    postgresql-42.x.x.jar
```

4.2.2 Node.js

If you have developed your project using Node.js as the server engine, Docker images with Node.js preinstalled are also available. You will probably need to do some additional configuration according to your system requirements, but the main steps are the same:

1. Create a “node” folder, and copy into it all the content of your application, including the code and all necessary modules.

2. Create a Dockerfile file with the instructions to create the image:

```
FROM node:20

WORKDIR /home/node/app

COPY . /home/node/app

CMD npm start
```

i Note

We use Node.js 20 LTS (Long Term Support) which is the current stable version. You can check for the latest LTS version on [Node.js official website](#).

For more options, search in the official image specification on [Docker Hub: node](#).

4.3 Database Setup

4.3.1 Local Database (PostgreSQL)

This section explains how to prepare the material to build our customized PostgreSQL image.

4.3.1.1 Why Copy the Database?

In production, you would typically use scripts to create the database schema and insert data. For this practice, we will copy the entire database data directory instead. This approach is simpler and faster at runtime since all data is already present.

4.3.1.2 Steps to Prepare the Database

1. **Stop the PostgreSQL server.** You must stop PostgreSQL before copying the data directory. Copying a running database can result in corrupted or inconsistent data.

```
sudo systemctl stop postgresql
```

 Warning

Make sure to stop PostgreSQL before copying the data directory. Copying a running database can result in corrupted data.

2. **Create the postgres folder** in your Docker folder:

```
cd $HOME/eclipse-workspace/sisinf_project/Docker
mkdir -p postgres
```

3. **Copy the database data directory.** The exact path depends on your PostgreSQL version and installation:

```
cd postgres
sudo cp -p -r /var/lib/postgresql/12/main .
sudo chown -R $USER:$USER main
```

 Tip

The path `/var/lib/postgresql/12/main` is typical for PostgreSQL 12 on Linux. Adjust the version number (12) and path according to your installation. You can find your PostgreSQL data directory by checking the `data_directory` setting in your PostgreSQL configuration.

4. **Compress the data directory** (optional but recommended, as it can be large):

```
cd postgres
zip -r main.zip main/
```

Checkpoint: PostgreSQL Preparation

At this point, you should have:

- A `postgres` folder inside your `Docker` folder
- Either a `main` directory or `main.zip` file containing your database data

Your folder structure should look like:

```
Docker/
  tomcat/
  ...
  postgres/
    main.zip  (or main/ directory)
```

4.3.2 Cloud Database (Option 2)

You can also deploy the database on a cloud server. Several cloud providers offer free databases under specific conditions. In addition to Google or Amazon, you can use:

- [Atlas](#) (for MongoDB)
- [Clever Cloud](#) (for PostgreSQL or MongoDB)

If you choose this solution:

- The Docker container for the database is **not needed**.
- You **must** configure your application server to connect to the database on the cloud.
- You **must** clearly indicate this choice in your practice report.

! Important

If using a cloud database, update your `server.xml` or `ConnectionManager` to use the cloud database connection string instead of “sisinf-database”. The connection string will be provided by your cloud database provider.

5 Image Construction and Test of the System

To build a Docker image, we need a `Dockerfile` file with its construction instructions. That file must be in the directory where we have all the necessary elements for each of the images.

5.1 Prerequisites

Before building images, ensure you have completed the preparation steps in Section 4:

- All files are in the correct folders
- Configuration files are properly modified
- Database data is ready (if using local database)

5.2 Building the Application Server Image

5.2.1 Tomcat Image

Create a file called `Dockerfile` in the folder `Docker/tomcat` with the following content:

```
FROM bitnami/tomcat:latest

LABEL maintainer="<your_name>"

COPY *.jar /opt/bitnami/tomcat/lib/

COPY context.xml /opt/bitnami/tomcat/conf/

COPY server.xml /opt/bitnami/tomcat/conf/

COPY ROOT.war /opt/bitnami/tomcat/webapps/
```

Explanation of Dockerfile instructions:

- `FROM bitnami/tomcat:latest` — identifies the base image (a pre-configured Tomcat image). Using `:latest` ensures you get the most recent stable version.
- `LABEL maintainer="<your_name>"` — identifies the author/maintainer of the image (replace `<your_name>` with your name or group identifier)
- `COPY *.jar /opt/bitnami/tomcat/lib/` — copies all JAR files (including the PostgreSQL driver) to Tomcat's library folder. `COPY` is preferred over `ADD` for simple file copying.
- `COPY context.xml /opt/bitnami/tomcat/conf/` — copies the context configuration file
- `COPY server.xml /opt/bitnami/tomcat/conf/` — copies the server configuration file (with database connection settings)
- `COPY ROOT.war /opt/bitnami/tomcat/webapps/` — copies the WAR file to the webapps folder for deployment

Tip

We use the Bitnami Tomcat image because it provides a well-configured, production-ready Tomcat installation. The paths (`/opt/bitnami/tomcat/`) are specific to this image.

5.2.1.1 Building the Image

To build the image, execute the following command in the folder that contains the Dockerfile:

```
cd $HOME/eclipse-workspace/sisinf_project/Docker/tomcat
sudo docker build -t sisinf/tomcat:latest .
```

! Important

The dot (.) at the end of the command is important — it tells Docker to use the current directory as the build context.

Expected output:

```
Sending build context to Docker daemon...
Step 1/5 : FROM bitnami/tomcat:latest
latest: Pulling from bitnami/tomcat
...
Step 2/5 : LABEL maintainer="<your_name>"
---> Running in [container_id]
...
Successfully built [image_id]
Successfully tagged sisinf/tomcat:latest
```

Verification: Image Built

After building, verify the image was created:

```
docker images | grep sisinf/tomcat
```

You should see `sisinf/tomcat` with tag `latest` in the list.

5.2.2 Node.js Image

If you're using Node.js, create a `Dockerfile` in the `node` folder:

```
FROM node:15

WORKDIR /home/node/app

COPY . /home/node/app

CMD npm start
```

Build the image:

```
cd $HOME/eclipse-workspace/sisinf_project/Docker/node
sudo docker build -t sisinf/node:latest .
```

5.3 Building the Database Image

Create a file called `Dockerfile` in the folder `Docker/postgres` with the following content:

```
FROM bitnami/postgresql:latest

LABEL maintainer="<your_name>"

COPY --chown=1001:1001 main /bitnami/postgresql/data
```

Explanation:

- `FROM bitnami/postgresql:latest` — uses a pre-configured PostgreSQL image. Using `:latest` ensures you get the most recent stable version compatible with your data.
- `LABEL maintainer="<your_name>"` — identifies the maintainer
- `COPY --chown=1001:1001 main /bitnami/postgresql/data` — copies the database data directory and sets the correct ownership (user 1001 is the PostgreSQL user in the Bitnami image). `COPY` is preferred over `ADD` for simple file copying.

! Important

If you compressed the database data, you must unzip it before building the image:

```
cd $HOME/eclipse-workspace/sisinf_project/Docker/postgres
unzip main.zip
```

Build the image:

```
cd $HOME/eclipse-workspace/sisinf_project/Docker/postgres
sudo docker build -t sisinf/postgresql:latest .
```

Expected output:

```
Sending build context to Docker daemon...
Step 1/3 : FROM bitnami/postgresql:latest
latest: Pulling from bitnami/postgresql
...
Successfully built [image_id]
Successfully tagged sisinf/postgresql:latest
```

5.4 Running the Containers

With these operations, we have created the images, but not the containers. To test that everything works correctly, we should create and start both containers.

5.4.1 Starting the Database Container

First, create a custom Docker network for container communication:

```
docker network create sisinf-network
```

Then, start the database on this network:

```
sudo docker run --name sisinf-database \
  --network sisinf-network \
  -e ALLOW_EMPTY_PASSWORD=yes \
  -p 5432:5432 \
  -d sisinf/postgresql:latest
```

Command explanation:

- `--name sisinf-database` — gives a name to the container
- `-e ALLOW_EMPTY_PASSWORD=yes` — allows the container to start even if the database has a password defined (avoids startup errors)
- `-p 5432:5432` — maps port 5432 of the container to port 5432 of the host machine (allows access from host via `localhost:5432`)
- `-d` — runs the container in daemon mode (in the background)
- `sisinf/postgresql:latest` — the image name to use

Tip

Verify the database container is running:

```
docker ps | grep sisinf-database
```

You should see the container listed with status “Up”.

5.4.2 Starting the Application Server Container

Create a custom Docker network

To allow containers to communicate, create a custom network first:

```
# Create a custom network
docker network create sisinf-network

# Start database on the network
docker run -d --name sisinf-database \
  --network sisinf-network \
  -e ALLOW_EMPTY_PASSWORD=yes \
  -p 5432:5432 \
  sisinf/postgresql:latest

# Start Tomcat on the same network
docker run -d --name sisinf-tomcat \
  --network sisinf-network \
  -p 8080:8080 \
  sisinf/tomcat:latest
```

Command explanation:

- `-d` — runs in daemon mode
- `--name sisinf-tomcat` — names the container
- `--network sisinf-network` — connects the container to a custom network (allows containers to communicate by name)
- `-p 8080:8080` — maps port 8080 of the container to port 8080 of the host
- `sisinf/tomcat:latest` — the image name

! Important

Using a custom Docker network is the recommended approach. Containers on the same network can communicate using their container names as hostnames. This allows Tomcat to access the database using “sisinf-database:5432” without needing deprecated flags.

5.4.3 Verifying the Deployment

After starting both containers:

1. **Check container status:**

```
docker ps
```

Both containers should be listed and running.

2. **Check application logs:**

```
docker logs sisinf-tomcat
```

Look for messages indicating Tomcat has started successfully.

3. **Access the application:**

Open your browser and navigate to:

`http://localhost:8080`

You should see your application's home page.

Tip

If you see errors in the logs or the application doesn't load, check:

- Database container is running: `docker ps | grep sisinf-database`
- Tomcat can reach the database: check the connection URL in `server.xml`
- Ports are not already in use: `netstat -an | grep 8080` or `netstat -an | grep 5432`

5.5 Creating a Deployment Script

Once everything works correctly, create a script that automates the deployment. Place this script in the `Docker` directory.

The script must:

1. Stop and remove existing containers (if they exist)
2. Build the images
3. Start the containers in the correct order

Here's an example script:

```
#!/bin/bash

# Create network if it doesn't exist
docker network create sisinf-network 2>/dev/null || true

# Stop and delete the previous version of the database, if it exists
docker stop sisinf-database 2>/dev/null
docker rm sisinf-database 2>/dev/null

# Build and start the database
cd postgres
if [ -f main.zip ]; then
    unzip -q main.zip
fi
sudo docker build -t sisinf/postgresql:latest .
sudo docker run --name sisinf-database \
    --network sisinf-network \
    -e ALLOW_EMPTY_PASSWORD=yes \
    -p 5432:5432 \
    -d sisinf/postgresql:latest
cd ..

# Wait for database to be ready
echo "Waiting for database to be ready..."
sleep 5

# Stop and delete the previous version of Tomcat, if it exists
docker stop sisinf-tomcat 2>/dev/null
docker rm sisinf-tomcat 2>/dev/null

# Build and start Tomcat
cd tomcat
```

```
sudo docker build -t sisinf/tomcat:latest .
docker run -d --name sisinf-tomcat \
  --network sisinf-network \
  -p 8080:8080 \
  sisinf/tomcat:latest
cd ..

echo "Deployment complete!"
echo "Application should be available at http://localhost:8080"
```

Make the script executable:

```
chmod +x deploy.sh
```

Checkpoint: Deployment Script

Your Docker folder structure should now be:

```
Docker/
  deploy.sh
  tomcat/
    Dockerfile
    ROOT.war
    context.xml
    server.xml
    postgresql-42.x.x.jar
  postgres/
    Dockerfile
    main.zip (or main/)
```

5.6 Test of the System

Before submitting the practice, you must validate that everything works correctly.

5.6.1 Testing Procedure

1. **On a clean system** (or after deleting all existing files, folders, and Docker containers):

```
# Remove existing containers
docker stop sisinf-database sisinf-tomcat 2>/dev/null
docker rm sisinf-database sisinf-tomcat 2>/dev/null

# Remove existing images (optional, for a truly clean test)
docker rmi sisinf/tomcat:latest sisinf/postgresql:latest
2>/dev/null
```

2. Run the deployment script:

```
cd $HOME/eclipse-workspace/sisinf_project/Docker
./deploy.sh
```

3. Verify the application works:

- Check containers are running: `docker ps`
- Access the application: `http://localhost:8080`
- Test key functionality (login, data display, etc.)

! Important

You must test the deployment on **at least two different computers** to ensure everything works correctly and the deployment is truly portable.

5.6.2 Common Issues and Solutions

⚠ Warning

Port already in use

If you get an error like “port 8080 is already in use”:

```
# Find what's using the port
sudo lsof -i :8080
# Or
sudo netstat -tulpn | grep 8080

# Stop the conflicting service or use a different port
```

Container name already exists

If you get “container name already exists”:

```
docker rm -f sisinf-database sisinf-tomcat
```

Database connection errors

If Tomcat cannot connect to the database:

- Verify the database container is running: `docker ps | grep sisinf-database`
- Check the connection URL in `server.xml` uses “sisinf-database” not “local-host”
- Check container logs: `docker logs sisinf-database` and `docker logs sisinf-tomcat`

6 Practice submission

The practice will be completed in groups of three. Submit a file named `practice5_NIA1_NIA2_NIA3.tar` or `practice5_NIA1_NIA2_NIA3.zip` (where NIA1, NIA2, and NIA3 are the Student Identification Numbers) with the following contents:

1. A text file named `autores.txt` containing in the first lines the NIA, surnames, and names of the authors. For example:

NIA	Apellidos	Nombre
545689	Rodríguez Quintela,	Sabela
745689	Caamaño Cives,	Antón
677893	López Marín,	Adrián

2. A ZIP file containing your work folder (Docker). This directory must include the deployment script and all necessary files:
 - `tomcat/` folder with configuration files, Dockerfile, database driver, and WAR file (or `node/` folder with Node.js application content)
 - `postgres/` folder with the database data (if using local database)

The deployment script must allow deploying and testing the application on any system with Docker installed, requiring only the execution of the script.

3. A file called `memoriaPractica5.pdf`, whose length will not exceed 20 pages, with the following sections:
 - Objective and final functional scope of the application (updated from Practice 1)
 - System approach and storyboard of the application (updated from Practice 2)
 - Data model of the system (updated from Practice 3)
 - Differences between the version presented in Practice 1 and the final version of the application

- Procedure for the deployment and installation of the application
- Information necessary for using the application: accounts, passwords, authentication methods, etc.
- Approximate schedule indicating efforts dedicated to development, and group assessment. What was most difficult? What have you learned? What would you have liked to implement but could not?

The report content (except the last four points) corresponds largely to documentation from previous practices, but updated to reflect the actual development. The functional scope, storyboard design, and data model may have changed from the initial design to the final version.

When unzipping the archive, the files and directories must be extracted into `practice5_NIA1_NIA2_NIA3/`. Follow the naming conventions and structure exactly.

6.1 Deadline and submission channel

The `.tar` or `.zip` file must be submitted via the University of Zaragoza's [Moodle](#) platform. The submission deadline is the day before the next lab session at 11:59 PM. If submission through ADD is not possible, send the archive to the course teachers by email.

7 Grading criteria

Once the practice is completed and submitted, each group must present it to the practice teachers in the session corresponding to the beginning of Practice 6. During the presentation, teachers will ask questions about the decisions made by the group.

The practice must be submitted as indicated above, and the application must work correctly on the teachers' computers after executing the deployment script provided by each group.

7.1 Minimum requirements (7 points)

- **Correct operation of the deployment scripts, and correct startup of the application.** (1 point)
- **Global operation of the application, without execution errors, and with visible operational functionality** (that is, that all parts of the application work correctly and none are unused or non-functional). (3 points)
- **Global aspect of the application. Interface design, accessibility and usability. Functional scope.** (1 point)

- **Content and presentation of the report and documentation of the information system.** (2 points)

7.2 Additional considerations (3 points)

- **Assessment of the application as an “information system”**, that is, its capacity to manage and communicate useful information to users, according to the objective intended when building the system. (2 points)
- **Originality, novel and differential aspects** that have been considered in the project as a whole. (1 point)
- Aspects related to the presentation of the report (structure, wording, punctuation, index, references and bibliography...), justification of the statements included in it and inclusion of a section on methodology and costs for the preparation of the practice.

8 Appendix. Optional: Advanced Deployment with Docker Compose and NGINX

This optional section shows how to enhance your deployment using Docker Compose and add an NGINX reverse proxy for improved security.

8.1 Why Use Docker Compose?

Docker Compose automates the management of multi-container applications. It provides several advantages:

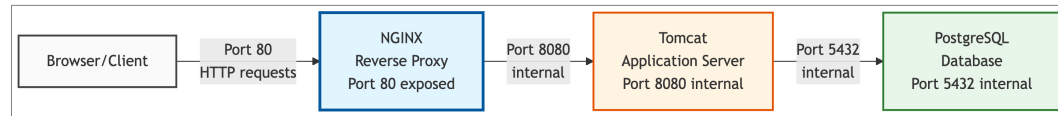
- Configure all components in a single `docker-compose.yml` file
- Automatically handle networking between containers
- Ensure containers start in the correct order
- Manage all services with simple commands

To use Docker Compose, you must first install it. See [Docker Compose Installation](#) for instructions.

8.2 Enhanced Architecture with NGINX

You can enhance your system by adding an NGINX reverse proxy. This improves security by hiding internal ports from external access. NGINX acts as the entry point, forwarding requests to Tomcat, which in turn connects to the database.

The following diagram shows the architecture:



Architecture explanation:

- **NGINX** acts as a reverse proxy and is the only service exposed to the internet (port 80). It receives all HTTP requests and forwards them to Tomcat.
- **Tomcat** runs the web application on port 8080 internally. It is not directly accessible from the internet, only through NGINX.
- **PostgreSQL** stores the data on port 5432 internally. It is not accessible from the internet, only by Tomcat.

This architecture improves security by hiding internal services. Only NGINX is exposed, and it forwards requests to Tomcat, which connects to the database.

8.3 Implementation Steps

8.3.1 1. Create NGINX Configuration

Create an `nginx` folder in your Docker directory and add an `nginx.conf` file:

```
http {
    server {
        listen 80;

        location / {
            proxy_pass http://tomcat:8080/;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
        }
    }
}
```

```
events {  
    worker_connections 1024;  
}
```

! Important

Note that the `proxy_pass` uses `http://tomcat:8080/` (the service name from docker-compose), not `http://sisinf-tomcat:8080/`. Docker Compose automatically resolves service names within the network.

8.3.2 2. Create NGINX Dockerfile

Create a Dockerfile in the `nginx` folder:

```
FROM nginx:latest  
  
COPY nginx.conf /etc/nginx/nginx.conf  
  
EXPOSE 80
```

8.3.3 3. Create docker-compose.yml

Create a `docker-compose.yml` file in your Docker directory:

```
version: '3.8'  
  
services:  
  database:  
    build:  
      context: ./postgres  
      dockerfile: Dockerfile  
    container_name: sisinf-database  
    environment:  
      - ALLOW_EMPTY_PASSWORD=yes  
    networks:  
      - backend  
    healthcheck:  
      test: ["CMD-SHELL", "pg_isready -U postgres"]  
      interval: 10s  
      timeout: 5s
```

```

    retries: 5

tomcat:
  build:
    context: ./tomcat
    dockerfile: Dockerfile
  container_name: sisinf-tomcat
  depends_on:
    database:
      condition: service_healthy
  networks:
    - frontend
    - backend

nginx:
  build:
    context: ./nginx
    dockerfile: Dockerfile
  container_name: sisinf-nginx
  ports:
    - "80:80"
  depends_on:
    - tomcat
  networks:
    - frontend

networks:
  frontend:
    driver: bridge
  backend:
    driver: bridge

```

Key points:

- **Only port 80** is exposed to the host (NGINX). Tomcat and PostgreSQL ports are internal only.
- **Service dependencies** ensure containers start in the correct order (database first, then Tomcat, then NGINX)
- **Health checks** ensure the database is ready before Tomcat starts
- Containers communicate using service names (**tomcat**, **database**) defined in the docker-compose.yml file

8.3.4 4. Update Tomcat Configuration

Update your `server.xml` in the Tomcat folder to use the service name `database` (not `sisinf-database`) since Docker Compose uses service names:

```
url="jdbc:postgresql://database:5432/postgres?currentSchema=<_
your_schema>"
```

8.3.5 5. Deploy with Docker Compose

Deploy the complete stack:

```
cd $HOME/eclipse-workspace/sisinf_project/Docker
docker-compose up -d --build
```

Expected output:

```
Creating network "docker_backend" ... done
Creating network "docker_frontend" ... done
Building database ...
Building tomcat ...
Building nginx ...
Creating sisinf-database ... done
Creating sisinf-tomcat ... done
Creating sisinf-nginx ... done
```

8.3.6 6. Access the Application

Access your application through NGINX:

`http://localhost`

(Note: port 80, not 8080)

8.4 Managing the Deployment

View all services:

```
docker-compose ps
```

View logs:

```
docker-compose logs -f
```

Stop all services:

```
docker-compose down
```

Restart a specific service:

```
docker-compose restart nginx
```

8.5 Benefits of This Architecture

- **Security:** Internal ports (8080, 5432) are not exposed to the internet
- **Flexibility:** Easy to add SSL/TLS certificates to NGINX for HTTPS
- **Scalability:** Can add load balancing or multiple Tomcat instances behind NGINX
- **Maintainability:** All configuration in one `docker-compose.yml` file

Tip

This advanced configuration is optional. The basic deployment with shell scripts (as described in the main sections) is sufficient to complete the practice. This enhanced setup is worth 1 additional point and demonstrates advanced Docker networking concepts.

9 Additional resources

- [Docker Get Started](#)
- [Docker Hub](#)
- [Docker Documentation](#)
- [Docker Compose Installation](#)
- [Docker Compose File Reference](#)
- [Node.js Docker Image](#)
- [PostgreSQL Docker Image](#)
- [NGINX Docker Image](#)

- [Atlas \(MongoDB Cloud\)](#)
- [Clever Cloud \(Database Hosting\)](#)
- [PostgreSQL JDBC Driver](#)